Forest cover classification with dense neural networks

Group 8 | Deep Learning Finals Project | 06 November 2026

We trained nine dense-network configurations on the full Forest Cover Type dataset and selected R1 using validation macro-F1. The frozen model achieved 88.75% test accuracy and 0.8315 macro-F1. This report evaluates a held-out sample from the source population; it does not establish performance in a new forest.

1. Dataset and question

The task is to predict one of seven forest cover types from 54 land attributes. Each observation represents a 30 x 30 metre cell in four wilderness areas of Roosevelt National Forest, northern Colorado. USFS records supplied the labels; USGS and USFS information supplied the cartographic inputs. The dataset contains 581,012 rows and no remotely sensed image inputs (Blackard, 1998, 1999).

Input group	Columns	Treatment
Elevation, aspect, slope, distances and hillshade	10 numeric	Fit scaling on training rows only
Wilderness designation	4 binary	Preserve 0/1 encoding
Soil / ecological landtype designation	40 binary	Preserve 0/1 encoding

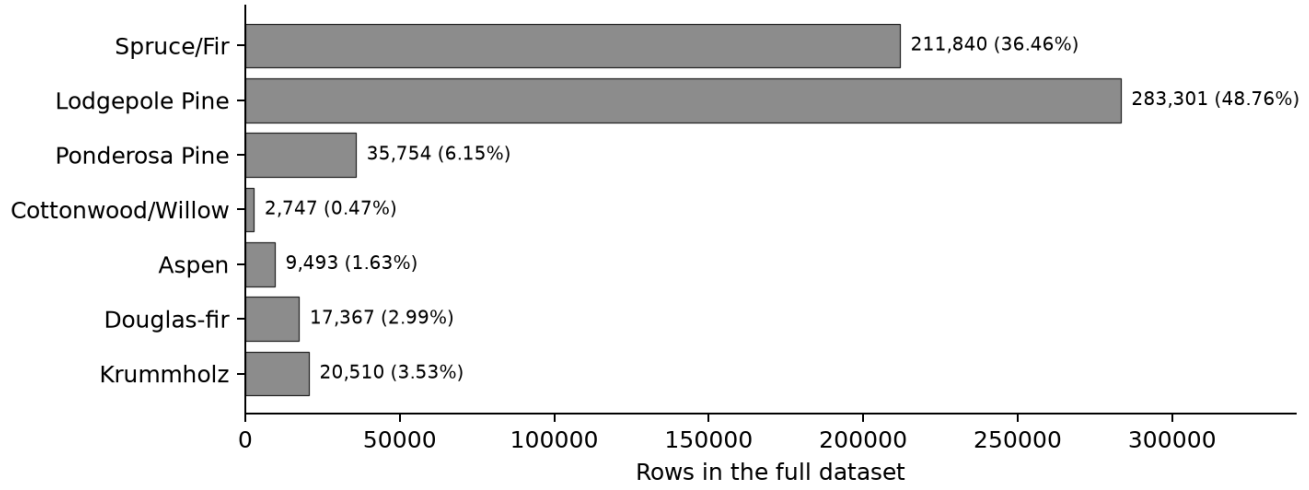


Figure 1. Counts from the loaded full dataset. Spruce/Fir and Lodgepole Pine account for 85.22% of rows; Cottonwood/Willow accounts for 0.47%.

2. Preprocessing and split

We mapped labels 1-7 to 0-6, checked all values for missingness/non-finite values, and verified the existing binary encoding. The checks found no missing/non-finite values and no repeated feature-vector hashes. A 64-bit hash check does not establish physical cell identity. Fixed stratified splits reserved 464,809 rows for training, 58,101 for validation and 58,102 for test (seeds 42 and 43). Split row IDs are disjoint; no feature hash was shared across partitions.

We fitted StandardScaler on the training subset's first ten columns and applied it to those columns in each partition (scikit-learn developers, n.d.-b). We kept the final 44 columns unchanged. Validation and test retain natural class proportions. R7 tests weights computed from training labels, rather than discarding most rows to match the smallest class.

Reproducibility and AI assistance

Execution used Python 3.12.16, TensorFlow 2.21.0, Keras 3.15.1, NumPy 2.5.4 and scikit-learn 1.9.1 on native Windows CPU. We fixed seeds, deterministic operations and thread settings. Nine runs took 9.0 minutes, excluding the pilot and reporting. Saved models, histories and split indices accompany the notebook; hardware or package changes can alter results. One seed does not quantify run-to-run uncertainty.

An AI assistant prepared the research, code, execution, analysis and report draft. Group members must check the work, understand the code and follow the instructor's AI-use rules. This report does not claim completed human contributions or private peer evaluations.

3. Model and controlled experiments

The baseline uses 54 inputs, two 50-unit ReLU layers and seven softmax outputs (5,657 parameters). We used Adam at learning rate 0.001, sparse categorical cross-entropy, batch 100, a 50-epoch cap, and early stopping on validation loss with patience 2 and best-weight restoration (Keras, n.d.-b, n.d.-c). Each run starts with fresh model, optimizer and callback state.

Each variant below changes one factor relative to R0. All runs share the full dataset, fixed splits, seed policy and epoch cap. The first seven rows cover the baseline plus six required changes; R7 and R8 investigate imbalance and regularization. Scores refer to the restored checkpoint. Epochs show executed / restored.

Run	Change from R0	Epochs	Val. loss	Val. acc.	Macro-F1
R0	Baseline	16 / 14	0.3417	85.75%	0.7827
R1	Width: 100-100	13 / 11	0.2793	88.66%	0.8272
R2	Depth: 50-50-50	20 / 18	0.2889	88.38%	0.8163
R3	Activation: tanh	34 / 32	0.2682	89.31%	0.8254
R4	Batch size: 500	22 / 20	0.3611	85.26%	0.7628
R5	Learning rate: 0.0003	42 / 40	0.3433	85.87%	0.7753
R6	Patience: 5	50 / 46	0.2901	88.21%	0.8270
R7	Balanced training weights	15 / 13	0.4965	79.59%	0.7353
R8	Dropout: 0.2	24 / 22	0.4185	82.51%	0.7114

Selection rule, fixed before training: highest validation macro-F1, then accuracy, lower loss, fewer parameters and run ID. Macro-F1 gives each class equal weight. Early stopping chooses a checkpoint within a run by loss; the selection rule compares those saved checkpoints across runs.

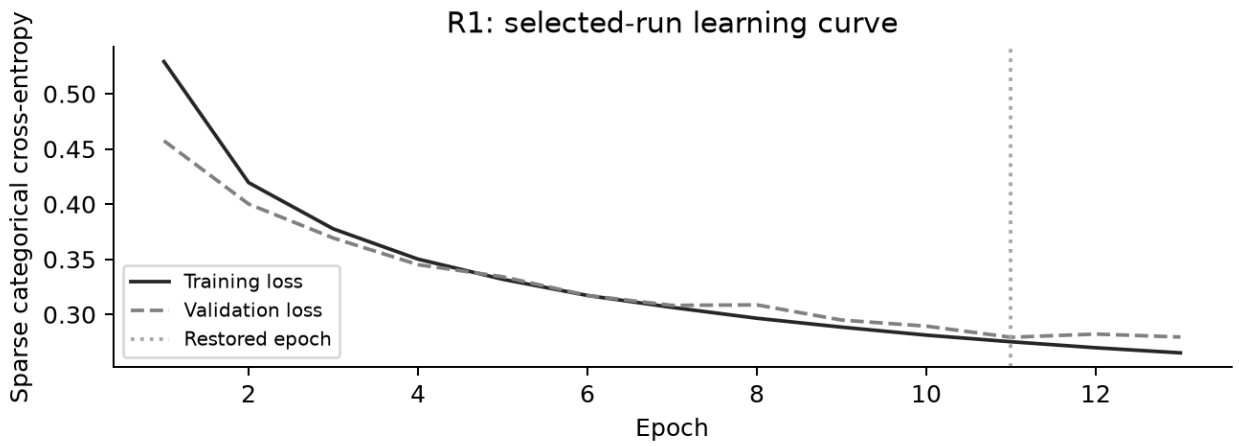


Figure 2. Loss history for R1; the dotted line marks restored epoch 11. Training loss is unweighted; validation loss is unweighted.

We selected R1 (Width: 100-100), architecture 54 - 100 - 100 - 7, with 16,307 parameters. Its validation accuracy was 88.66%; macro-F1 was 0.8272. R1 led R6 by only 0.0002 macro-F1 in this single-seed comparison. 1 of nine runs reached the epoch cap; we did not test whether further training would improve those runs. The notebook preserves histories, configurations and model hashes.

Additional techniques: measured benefit or cost

Balanced training weights use n_train / (7 x class_count_train). R7 changed validation macro-F1 from 0.7827 to 0.7353, accuracy from 85.75% to 79.59%, and mean class recall from 75.15% to 87.35%. Weighting improved class recall while reducing precision and aggregate accuracy under this comparison.

Dropout removes hidden activations during training and is disabled at inference (Keras, n.d.-a). R8 adds rate 0.2 after each hidden layer. Validation macro-F1 changed to 0.7114 (-0.0713 versus R0), with 82.51% accuracy. It did not improve the declared objective in this single-seed comparison. Bonus eligibility depends on whether the class had covered dropout.

4. Frozen-model results

After validation selected the model, we froze its weights, preprocessing and class map. We did not refit on training plus validation. One final model.evaluate call measured test loss and accuracy; predictions from the same frozen model supplied the class report, confusion matrices and mistakes. Normal notebook reruns load the saved final assessment.

Partition	Rows	Accuracy	Macro-F1	Balanced acc.
Train	464,809	89.29%	0.8368	83.66%
Validation	58,101	88.66%	0.8272	82.50%
Test	58,102	88.75%	0.8315	83.24%

True cover type	Test support	Precision	Recall	F1
Spruce/Fir	21,184	0.9078	0.8599	0.8832
Lodgepole Pine	28,331	0.8844	0.9277	0.9056
Ponderosa Pine	3,576	0.8589	0.8884	0.8734
Cottonwood/Willow	274	0.7310	0.9124	0.8117
Aspen	949	0.7327	0.6786	0.7046
Douglas-fir	1,737	0.8458	0.6442	0.7314
Krummholz	2,051	0.9059	0.9157	0.9108

Per-class recall is the proportion of a true class that the model predicted correctly, our definition of the brief's 'accuracy for each class'. Balanced accuracy averages these seven recalls. Macro-F1 averages the seven class F1 scores (scikit-learn developers, n.d.-a).

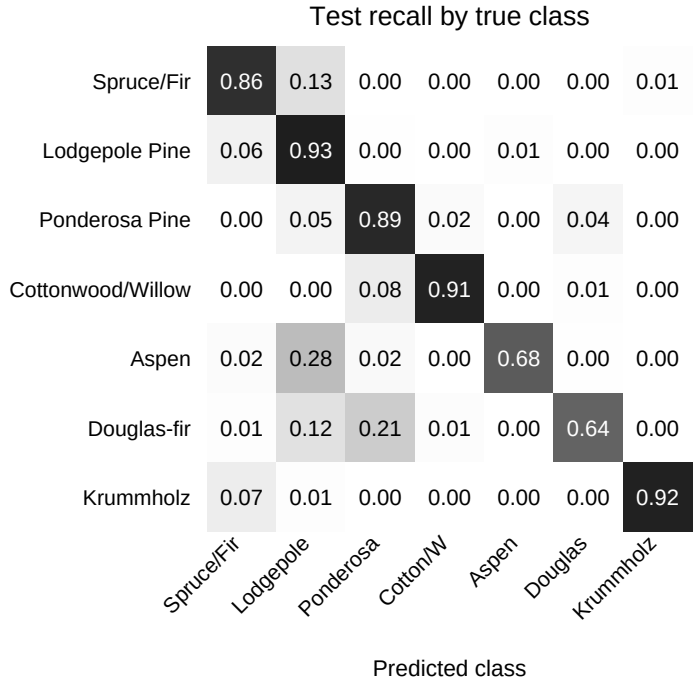


Figure 3. Row-normalized test confusion matrix: rows are true classes and columns are predictions. The diagonal gives class recall. Cotton/W denotes Cottonwood/Willow. The notebook also includes raw counts.

The frozen model's training accuracy exceeded validation accuracy by 0.64 percentage points. This descriptive gap is small on this split; one split cannot establish generalization to new regions. Lodgepole Pine had the highest recall (92.77%); Douglas-fir had the lowest (64.42%). The majority-class reference achieved 48.76% on these test labels, compared with the selected model's 88.75%.

5. Five wrong test examples

The final model misclassified 6,534 of 58,102 test cells. We selected the lowest-row-ID mistake from each available true class until five cases, filling any remaining places by row ID. We declared this rule before test analysis. The error table retains original measurement units; the notebook and CSV include the remaining features.

Row ID	True cover	Predicted cover	Elev. m	Slope	Max score
115	Spruce/Fir	Lodgepole Pine	3195	15 deg	67.65%
444	Lodgepole Pine	Spruce/Fir	3004	6 deg	70.27%
2844	Ponderosa Pine	Douglas-fir	2329	24 deg	69.95%
2180	Cottonwood/Willow	Ponderosa Pine	2030	18 deg	46.81%
45	Aspen	Lodgepole Pine	2529	8 deg	55.42%

Five deterministic error examples. 'Max score' is the largest softmax output; we did not calibrate it as a probability of correctness.

Observed context and possible explanations

Case 1, row 115. This cell and case 2 share Rawah and soil indicator 29 despite different true labels. Those two categorical inputs alone cannot separate these examples. Overlap in their land context is a plausible contributor to the Spruce/Fir-Lodgepole confusion.

Case 2, row 444. The reverse Spruce/Fir-Lodgepole error occurs here at 3,004 m, versus 3,195 m in case 1. Blackard (1999) lists both cover types in Rawah. The shared area and soil category may leave a boundary that this network did not resolve from the remaining measurements.

Case 3, row 2844. The cell has a recorded horizontal water distance of 0 m, slope of 24 degrees and soil indicator 10. Blackard (1999) lists both Ponderosa Pine and Douglas-fir among the main cover types in Cache la Poudre. Shared area-level conditions could contribute to this confusion; the water-distance value does not uniquely identify a cover type.

Case 4, row 2180. At 42 m horizontal distance from water, this cell received a Ponderosa Pine score of 46.81%. Training contained 2,198 Cottonwood/Willow cells and 28,603 Ponderosa Pine cells. That unequal representation may contribute, although the model's 91.24% test recall for Cottonwood/Willow shows that it recognized most members of the class.

Case 5, row 45. Aspen and Lodgepole Pine both occur in Rawah (Blackard, 1999). This cell has soil indicator 18, slope 8 degrees and elevation 2,529 m. Lodgepole Pine has 226,640 training examples versus 7,594 for Aspen. Shared area and unequal representation offer plausible explanations for the majority-class prediction.

What the examples establish

These rows establish specific classification errors and their recorded input context. We did not compute local feature attributions, inspect field conditions, or demonstrate a causal mechanism. The suggested explanations therefore remain hypotheses. A wrong prediction with a high softmax score also shows why confidence scores need separate calibration before users treat them as reliability estimates.

6. Trust and limits

We would use this model as a screening aid for source-like land records, with human verification and class-specific review. Test accuracy of 88.75% supports that limited use under this split. The weakest class recall of 64.42% and 6,534 observed errors rule out treating each predicted cover type as a verified field observation. We would not use this study alone to approve operational forest-management decisions.

The evaluation covers a random split from four Colorado wilderness areas. It does not test a new region, collection period, managed forest, or Philippine forest population. Nearby or ecologically related cells may share conditions across a random split. We did not measure spatial dependence; an unseen-area study would require grouped/geographic validation. The task predicts cover type, not wildfire risk, deforestation, or the species of every individual tree (Blackard, 1998, 1999).

References

Blackard, J. A. (1998). *Covertype* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C50K5N

Blackard, J. A. (1999, December 7). *The Forest CoverType dataset* [Data set documentation]. UCI Machine Learning Repository. https://archive.ics.uci.edu/ml/machine-learning-databases/covtype/covtype.info

Keras. (n.d.-a). *Dropout layer*. Retrieved November 6, 2026, from https://keras.io/api/layers/regularization_layers/dropout/

Keras. (n.d.-b). *EarlyStopping*. Retrieved November 6, 2026, from https://keras.io/api/callbacks/early_stopping/

Keras. (n.d.-c). *Model training APIs*. Retrieved November 6, 2026, from https://keras.io/api/models/model_training_apis/

scikit-learn developers. (n.d.-a). *classification_report*. scikit-learn. Retrieved November 6, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

scikit-learn developers. (n.d.-b). *Common pitfalls and recommended practices*. scikit-learn. Retrieved November 6, 2026, from https://scikit-learn.org/stable/common_pitfalls.html